SESIÓN N° 7

Tema: Conectividad a Bases de Datos con JDBC en Java

I. Objetivo de la Guía

Implementar la conectividad entre aplicaciones Java y bases de datos relacionales mediante la API JDBC y el uso de PreparedStatement para realizar operaciones CRUD seguras.

II. Marco Teórico Breve

1. Java Database Connectivity (JDBC)

- JDBC es la API estándar de Java para conectarse a bases de datos relacionales. El driver (MySQL Connector/J) traduce entre Java y MySQL. DriverManager elige el adaptador correcto según la URL (jdbc:mysql://...).
- Piensa en JDBC como un adaptador de enchufe universal: tu programa Java "habla" un idioma y cada motor de BD habla otro. El driver es el adaptador específico; DriverManager es quien elige el adaptador correcto según la URL.

2. Consultas Preparadas (PreparedStatement)

- PreparedStatement permite escribir SQL con signos de interrogación (?) como huecos seguros. Los valores se asignan después con setString, setInt, etc. Esto evita errores de sintaxis y ataques de inyección SQL.
- Piensa en PreparedStatement como un formulario impreso con espacios en blanco (los ?): la estructura ya está fija y tú solo rellenas los espacios con datos. Nadie puede "colar" código SQL malicioso en un espacio en blanco.

Errores Comunes al Empezar

Error típico	Cómo evitarlo
Escribir mal el host, el puerto o el nombre de la base de datos en la URL de conexión.	Revisa que MySQL esté encendido y que la URL tenga exactamente el mismo nombre de BD que creaste en la Sesión 6 (gamestore_db).
Concatenar valores del usuario dentro del SQL de un Statement en vez de usar PreparedStatement con ?.	Usa siempre PreparedStatement con ? para cualquier valor que venga de un formulario o del usuario.
Olvidar cerrar la Connection, o no usar try-with-resources.	Usa try-with-resources para que Java cierre la conexión automáticamente, incluso si algo falla.

III. Actividades Prácticas Paso a Paso

Parte 1

Desarrollo del Trabajo 1 (Clase de Conexión Centralizada)

Crea la clase ConexionBD.java que centralice los parámetros de conexión y exponga un método static obtenerConexion() para que el resto del proyecto obtenga una Connection lista para usar.

Código completo — ConexionBD.java:

```
import java.sql.Connection;
import java.sql.DriverManager;
```

```java
import java.sql.SQLException;

public class ConexionBD {
    private static final String URL =
"jdbc:mysql://localhost:3306/gamestore_db?useSSL=false&serverTimezone=UTC";
    private static final String USUARIO = "root";
    private static final String PASSWORD = "";   // cámbiala si tienes

    public static Connection obtenerConexion() {
        try {
            Connection conn = DriverManager.getConnection(URL, USUARIO, PASSWORD);
            System.out.println("¡Conexión exitosa!");
            return conn;
        } catch (SQLException e) {
            System.out.println("Error: " + e.getMessage());
            return null;
        }
    }

    // Prueba rápida
    public static void main(String[] args) {
        obtenerConexion();
    }
}
```

Explicación línea por línea (puntos clave):

import java.sql.Connection;
Importa Connection, la interfaz que representa una conexión abierta con la base de datos.
import java.sql.DriverManager;
Importa DriverManager, la clase que sabe fabricar una Connection a partir de una URL.
import java.sql.SQLException;
Importa SQLException, la excepción que hay que atrapar en cualquier operación con la base de datos.
public class ConexionBD {
Declara la clase ConexionBD.
private static final String URL =
"jdbc:mysql://localhost:3306/gamestore_db?useSSL=false&serverTimezone=UTC";
Constante URL: la dirección jdbc que indica el motor (mysql), el host (localhost), el puerto (3306) y la base de datos (gamestore_db) a la que conectarse.
private static final String USUARIO = "root";
Constante USUARIO: el nombre de usuario de MySQL con el que se inicia sesión.
private static final String PASSWORD = ""; // Cambiar según la configuración local
Constante PASSWORD: la contraseña de ese usuario (vacía aquí; debes cambiarla según tu instalación local de MySQL).
public static Connection obtenerConexion() {
Declara el método static obtenerConexion(), que devuelve un objeto Connection (o null si algo falla).
Connection conexion = null;
Declara la variable conexion, inicializada en null por si la conexión falla.
try {
Abre un bloque try normal (sin try-with-resources aquí, porque esta conexión se devuelve para que OTRA clase la use y la cierre después).
conexion = DriverManager.getConnection(URL, USUARIO, PASSWORD);

Intenta crear la conexión real usando DriverManager, pasándole la URL, el usuario y la contraseña.
System.out.println("¡Conexión exitosa a la base de datos MySQL!");
Si la conexión se logra, muestra en consola un mensaje de éxito.
} catch (SQLException e) {
Bloque catch que atrapa SQLException si la conexión falla (credenciales incorrectas, servidor apagado, etc.).
System.out.println("Error al conectar con la base de datos: " + e.getMessage());
Muestra en consola un mensaje de error con el detalle de la excepción.
}
Cierra el bloque catch.
return conexion;
Devuelve el objeto conexion: será un Connection válido si todo salió bien, o null si hubo un error.
}
Cierra el método obtenerConexion().
}
Cierra la clase ConexionBD.

Parte 2

Desarrollo del Trabajo 2 (DAO — Create y Read con PreparedStatement)

Crea la clase EstudianteDAO.java con métodos para insertar un estudiante (Create) y listar todos los estudiantes (Read), usando PreparedStatement y try-with-resources.

Código completo — EstudianteDAO.java:

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class EstudianteDAO {

    // Método para registrar un nuevo estudiante (Create - Trabajo 2)
    public void insertarEstudiante(String nombre, int puntos) {
        String sql = "INSERT INTO estudiantes (nombre, puntos) VALUES (?, ?)";

        try (Connection conn = ConexionBD.obtenerConexion();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, nombre);
            pstmt.setInt(2, puntos);
            pstmt.executeUpdate();
            System.out.println("Estudiante registrado exitosamente en la BD.");

        } catch (SQLException e) {
            System.out.println("Error al insertar estudiante: " + e.getMessage());
        }
    }

    // Método para listar estudiantes (Read - Trabajo 2)
    public void listarEstudiantes() {
        String sql = "SELECT id_estudiante, nombre, puntos FROM estudiantes";
```

```
        try (Connection conn = ConexionBD.obtenerConexion();
             PreparedStatement pstmt = conn.prepareStatement(sql);
             ResultSet rs = pstmt.executeQuery()) {

            System.out.println("--- LISTA DE ESTUDIANTES (DESDE MYSQL) ---");
            while (rs.next()) {
                int id = rs.getInt("id_estudiante");
                String nombre = rs.getString("nombre");
                int puntos = rs.getInt("puntos");
                System.out.println("ID: " + id + " | Nombre: " + nombre + " | Puntos: " +
puntos);
            }

        } catch (SQLException e) {
            System.out.println("Error al consultar estudiantes: " + e.getMessage());
        }
    }
}
```

Explicación línea por línea (puntos clave):

import java.sql.Connection;
Importa Connection.
import java.sql.PreparedStatement;
Importa PreparedStatement, para construir consultas SQL seguras con parámetros.
import java.sql.ResultSet;
Importa ResultSet, para recorrer los resultados de una consulta SELECT.
import java.sql.SQLException;
Importa SQLException.
public class EstudianteDAO {
Declara la clase EstudianteDAO (DAO = Data Access Object, un objeto dedicado a acceder a los datos de una tabla).
public void insertarEstudiante(String nombre, int puntos) {
Declara el método insertarEstudiante, que recibe el nombre y los puntos del nuevo estudiante.
String sql = "INSERT INTO estudiantes (nombre, puntos) VALUES (?, ?)";
Define la consulta SQL con DOS signos de interrogación (?): son los espacios donde se insertarán nombre y puntos de forma segura.
try (Connection conn = ConexionBD.obtenerConexion();
Abre un try-with-resources que declara DOS recursos: la Connection (obtenida de ConexionBD) y el PreparedStatement construido a partir de ella.
PreparedStatement pstmt = conn.prepareStatement(sql)) {
Continúa la declaración del PreparedStatement, creado con conn.prepareStatement(sql).
pstmt.setString(1, nombre);
Asigna el primer ? (posición 1) al valor de nombre, como texto.
pstmt.setInt(2, puntos);
Asigna el segundo ? (posición 2) al valor de puntos, como número entero.
pstmt.executeUpdate();
Ejecuta la consulta de escritura (INSERT) contra la base de datos.
System.out.println("Estudiante registrado exitosamente en la BD.");
Muestra en consola un mensaje confirmando que el estudiante se registró.
} catch (SQLException e) {
Bloque catch que atrapa SQLException si algo falla al insertar.

System.out.println("Error al insertar estudiante: " + e.getMessage());
Muestra en consola un mensaje de error con el detalle de la excepción.
}
Cierra el bloque catch.
}
Cierra el método insertarEstudiante (Java cierra el PreparedStatement y la Connection automáticamente, en orden inverso, gracias a try-with-resources).
public void listarEstudiantes() {
Declara el método listarEstudiantes, sin parámetros.
String sql = "SELECT id_estudiante, nombre, puntos FROM estudiantes";
Define la consulta SQL que selecciona las tres columnas de la tabla estudiantes.
try (Connection conn = ConexionBD.obtenerConexion();
Abre un try-with-resources que declara TRES recursos encadenados: Connection, PreparedStatement y (a continuación) ResultSet.
PreparedStatement pstmt = conn.prepareStatement(sql);
Continúa la declaración del PreparedStatement.
ResultSet rs = pstmt.executeQuery()) {
Declara el ResultSet, ejecutando la consulta de lectura (SELECT) con pstmt.executeQuery().
System.out.println("--- LISTA DE ESTUDIANTES (DESDE MYSQL) ---");
Muestra en consola un encabezado antes de listar los estudiantes.
while (rs.next()) {
Bucle while: rs.next() mueve el cursor a la siguiente fila del resultado y devuelve false cuando ya no hay más filas.
int id = rs.getInt("id_estudiante");
Lee la columna id_estudiante de la fila actual como número entero.
String nombre = rs.getString("nombre");
Lee la columna nombre de la fila actual como texto.
int puntos = rs.getInt("puntos");
Lee la columna puntos de la fila actual como número entero.
System.out.println("ID: " + id + " | Nombre: " + nombre + " | Puntos: " + puntos);
Muestra en consola los tres datos leídos de esa fila.
}
Cierra el bucle while.
} catch (SQLException e) {
Bloque catch que atrapa SQLException si algo falla al consultar.
System.out.println("Error al consultar estudiantes: " + e.getMessage());
Muestra en consola un mensaje de error con el detalle de la excepción.
}
Cierra el bloque catch.
}
Cierra el método listarEstudiantes.
}
Cierra la clase EstudianteDAO.

Clase de ejecución — PrincipalJDBC.java

Clase de prueba que inserta un estudiante y lista todos los registros desde MySQL:

Código completo:

```
public class PrincipalJDBC {
    public static void main(String[] args) {
        EstudianteDAO dao = new EstudianteDAO();
```

```
        // Probando la inserción conectada a MySQL (Trabajo 2)
        dao.insertarEstudiante("Lucía Pérez", 420);

        // Probando la lectura de registros (Trabajo 2)
        dao.listarEstudiantes();
    }
}
```

Explicación línea por línea (puntos clave):

public class PrincipalJDBC {
Declara la clase PrincipalJDBC, que probará la capa de persistencia completa.
public static void main(String[] args) {
Punto de entrada del programa: aquí comienza la ejecución cuando se corre esta clase.
EstudianteDAO dao = new EstudianteDAO();
Crea un objeto EstudianteDAO (a diferencia de ConexionBD, aquí SÍ hace falta un objeto porque sus métodos no son static).
dao.insertarEstudiante("Lucía Pérez", 420);
Llama a dao.insertarEstudiante(...) para guardar un nuevo estudiante directamente en la tabla estudiantes de MySQL.
dao.listarEstudiantes();
Llama a dao.listarEstudiantes() para leer y mostrar en consola todos los estudiantes guardados en la base de datos, incluido el que se acaba de insertar.
}
Cierra el método main.
}
Cierra la clase PrincipalJDBC.

Salida esperada en consola:

```
Estudiante registrado exitosamente en la BD.
¡Conexión exitosa!
--- LISTA DE ESTUDIANTES (DESDE MYSQL) ---
ID: 1 | Nombre: Carlos Mendoza | Puntos: 600
ID: 2 | Nombre: Ana Torres | Puntos: 350
ID: 3 | Nombre: Lucía Pérez | Puntos: 420
PS E:\Semestre IV\Taller de lenguajes de programacion\sesion 7>
```



Resumen de Conceptos Clave

- **DriverManager**: Abre la conexión real hacia la BD con getConnection(URL, usuario, password).
- **Connection**: Objeto que representa la conexión abierta y activa entre Java y MySQL.
- **PreparedStatement**: Consulta preparada con ? seguros; evita inyección SQL. Usar setString/setInt antes de executeUpdate/executeQuery.

- **ResultSet**: Contiene las filas devueltas por un SELECT; se recorre con while (rs.next()).
- **SQLException**: Excepción que se lanza cuando falla la comunicación con la base de datos.
- **DAO (Data Access Object):** Patrón que concentra las operaciones de persistencia (insertar, listar, etc.) en una clase dedicada.